Polynomial Regression

From Basics to Pro — A Visual, Hands-On Guide

Understand why straight lines fail on curved data, how to build polynomial models by hand and with scikit-learn, and how to pick the right degree without overfitting — explained step by step with math, code and charts.

1. What Is Polynomial Regression?
2. Quick Recap: Linear Regression
3. Why a Straight Line Isn't Enough
4. The Polynomial Regression Equation
5. Building It in Python (scikit-learn)
6. Choosing the Right Degree
7. Underfitting vs. Overfitting
8. The Bias–Variance Trade-off
9. Multiple-Feature Polynomial Regression
10. Finding the Best Degree (Pro Workflow)
11. The Math Behind It (Normal Equation)
12. Cheat Sheet & Key Takeaways

1 · What Is Polynomial Regression?

Polynomial regression is a way of teaching a model to fit a **curved** relationship between an input *x* and an output *y*, instead of forcing a straight line through the data. It is still built on top of ordinary linear regression — the trick is simply to feed the model extra input columns like x^2, x^3, x^4 (powers of x) alongside the original x. The model then finds the best weight for each of those columns, which lets the final prediction curve bend.

Think of it like this: a straight line has only two knobs to turn — slope and intercept. A polynomial curve has one extra knob per extra power of x, so it can bend, dip and rise to match more complicated patterns in real data (price growth, sensor drift, population curves, and so on).

> You will see this idea called “non-linear regression” in some tutorials — that's the same concept: the *relationship* is non-linear, even though the model is still solved using linear algebra underneath.

2 · Quick Recap: Linear Regression

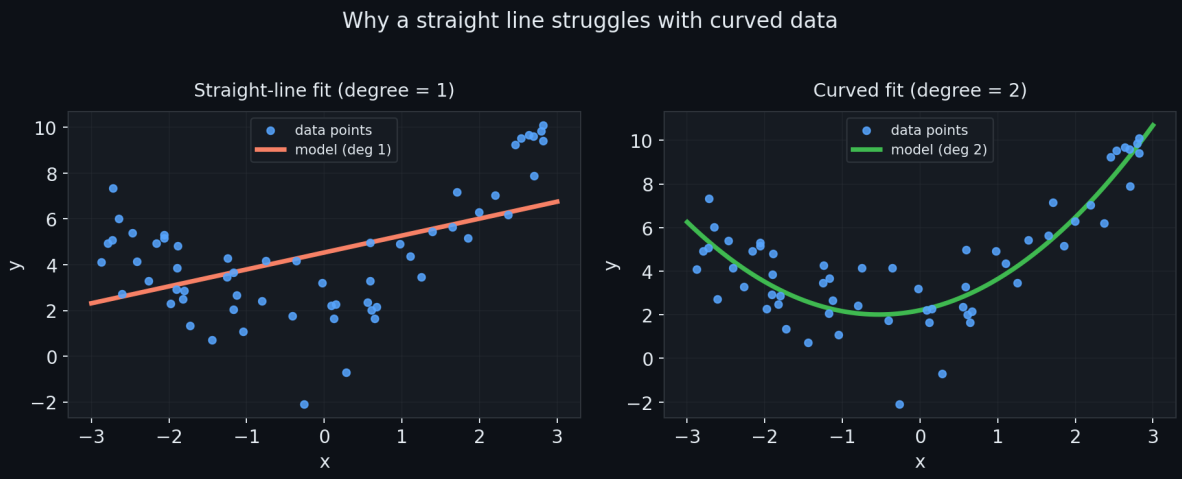
Before polynomial regression makes sense, it helps to remember plain linear regression. With three features (say, years of experience x_1, years at the company x_2, and base salary x_3), a linear regression model predicts the output $\hat{y}$ as a weighted sum:

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Here w_1, w_2, w_3 are the learned weights (how much each feature matters) and *b* is the bias (an offset). This equation can only ever draw a straight line (or, with multiple features, a flat plane). That is exactly its limitation.

3 · Why a Straight Line Isn't Enough

Real-world data is rarely a perfect straight line. If the true pattern curves — for example, growth that accelerates, or a cost that rises faster and faster — a straight-line model will systematically miss the shape no matter how well it is trained. This is called **underfitting**: the model is too simple to capture the pattern.



Left: a straight line cannot bend to follow curved data. Right: adding an x-squared term lets the model follow the curve.

Both charts are fit on the exact same data points. The only difference is that the right-hand model was given an extra input column, x^2, computed from the original x. That one addition is the entire idea behind polynomial regression.

4 · The Polynomial Regression Equation

For a single feature x, a polynomial regression model of degree *d* looks like this:

$$\hat{y} = w_1 x + w_2 x^2 + w_3 x^3 + \dots + w_d x^d + b$$

Notice this is still a *linear* equation in terms of the weights w — we're just feeding it non-linear *features* (x, x-squared, x-cubed …). That's why the same linear regression machinery (and the same normal-equation math) can be reused to solve it.

Degree	Model shape	Equation
1 (linear)	straight line	$\hat{y} = w_1 x + b$
2 (quadratic)	one bend / bowl	$\hat{y} = w_1 x + w_2 x^2 + b$
3 (cubic)	S-shaped curve	$\hat{y} = w_1 x + w_2 x^2 + w_3 x^3 + b$

Higher degree means more bends available, which means a more flexible curve. But, as you'll see in Section 7, more flexibility is not automatically better.

5 · Building It in Python (scikit-learn)

In practice you rarely type out x-squared, x-cubed by hand. scikit-learn's `PolynomialFeatures` class generates all the extra power columns for you, and a normal `LinearRegression` then learns the weights.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# X is your original single-column feature, shape (n_samples, 1)
poly = PolynomialFeatures(degree=2)     # degree = how many powers to add
X_poly = poly.fit_transform(X)          # turns [x] into [1, x, x^2]

model = LinearRegression()
model.fit(X_poly, y)                    # learn the weights

y_pred = model.predict(poly.transform(X_new))
```

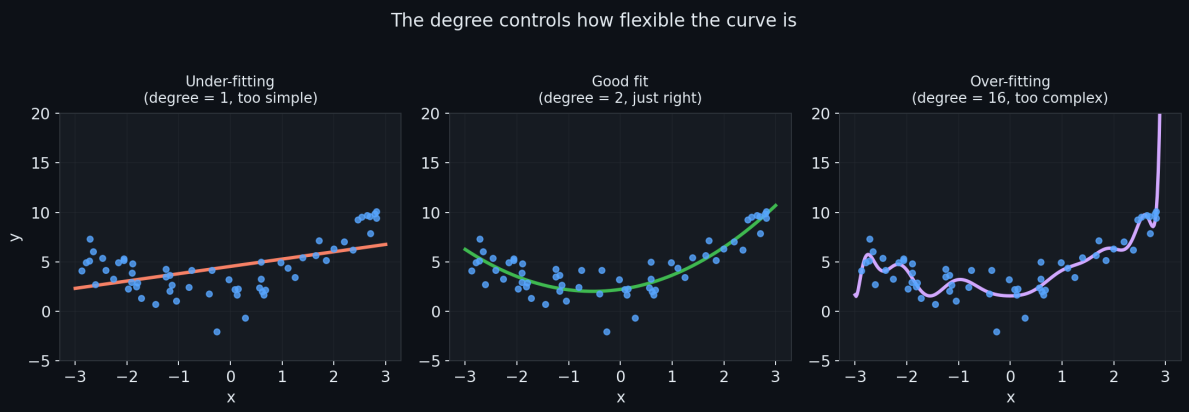
A few important rules of thumb when using `PolynomialFeatures`:

- Fit the transformer (`poly.fit_transform`) only on the training input, then reuse it (`poly.transform`) on the test input — never re-fit on test data.
- Only transform the input features (X). Never apply the polynomial transform to the target/output (y).
- `include_bias=True` (the default) adds a constant column of 1s for the intercept; set it to `False` if your model already fits an intercept separately.

For one input column, degree *d* produces *d* new feature columns ($x, x^2, \ldots, x^d$), plus an optional bias column.

6 · Choosing the Right Degree

The degree is the single most important knob in polynomial regression. Pick it too low and the model can't capture the pattern (underfitting). Pick it too high and the model starts memorizing noise instead of learning the real trend (overfitting).



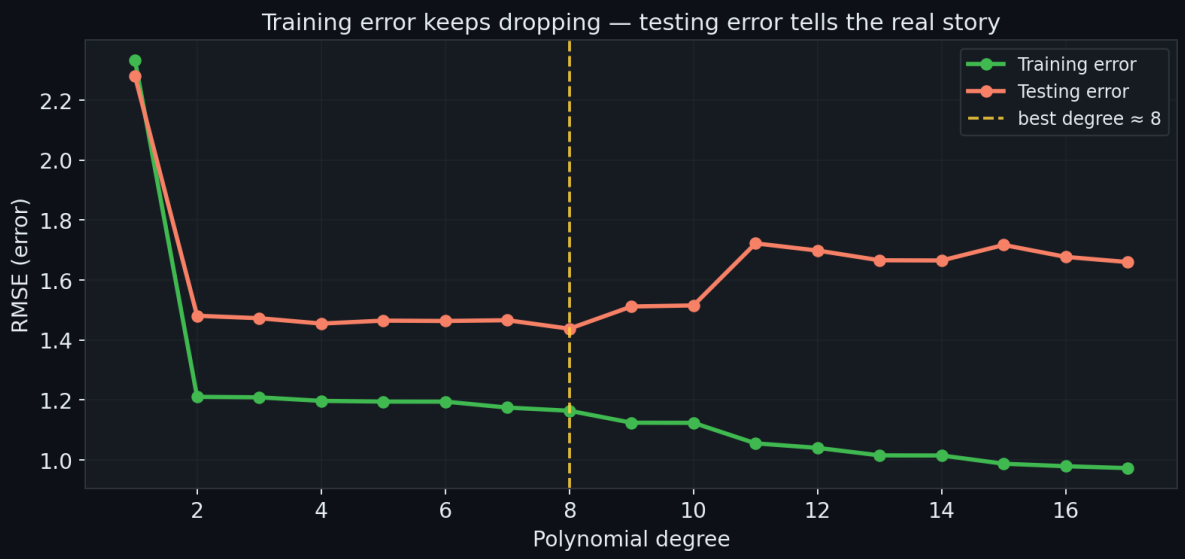
Same data, three different degrees. Degree 1 misses the curve; degree 2 matches it; degree 16 chases every noisy point.

In the right-most panel, the wiggly degree-16 curve passes very close to almost every training point — but that wiggliness has nothing to do with the real underlying pattern. It would predict poorly on any new data point.

7 · Underfitting vs. Overfitting

Underfitting	Good Fit	Overfitting
Degree too low	Degree “just right”	Degree too high
High error on training *and* test data	Low error on both training and test data	Very low training error, high test error
Model is too simple to see the pattern	Model captures the true trend	Model memorizes noise, doesn't generalize

The clearest way to detect over/underfitting is to compare training error against testing error while increasing the degree:



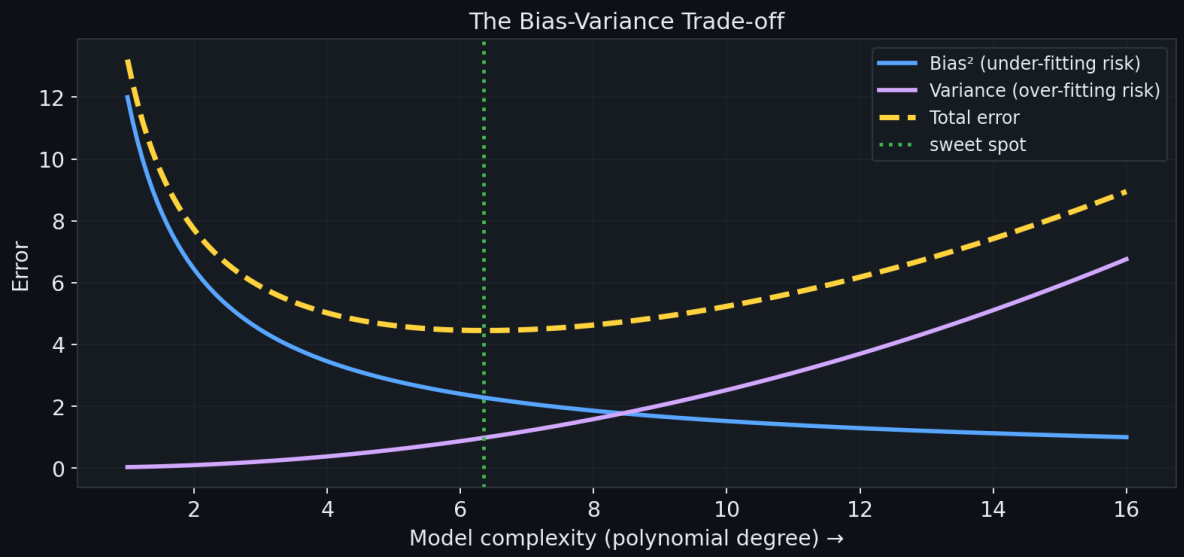
Training error (green) keeps falling as degree rises — but testing error (orange) bottoms out and then climbs back up. That turning point is the sweet spot.

Training error will almost always keep shrinking as you add more powers — that alone tells you nothing useful. What matters is the **testing error**: once it stops improving and starts rising again, you've gone past the ideal degree.

8 · The Bias–Variance Trade-off

Underfitting and overfitting are two ends of the same trade-off, usually described with two technical terms:

- **Bias** — error from a model that is too simple and makes systematic mistakes (high at low degrees).
- **Variance** — error from a model that is too sensitive to the exact training data, changing wildly with small data changes (high at high degrees).



As complexity (degree) rises, bias falls but variance rises. The best degree sits where their combined error is lowest.

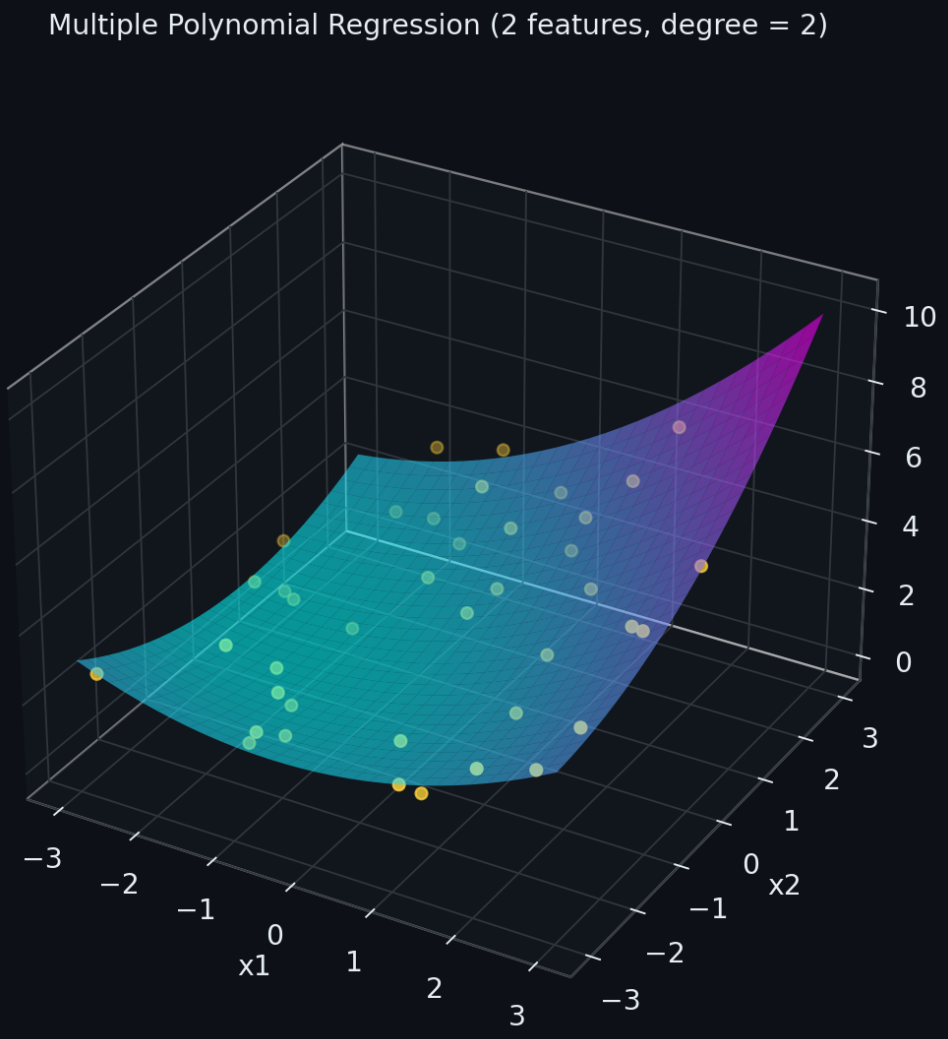
Every model-selection technique in this guide — comparing train/test error, cross-validation, and so on — is really just a practical way of finding that sweet spot on this curve.

9 · Multiple-Feature Polynomial Regression

Real datasets usually have more than one input feature. With two features x_1 and x_2, a degree-2 polynomial model doesn't just add x_1^2 and x_2^2 — it also adds the **interaction term** $x_1 x_2$, because that term also has total degree 2 (power of x1 + power of x2 = 2):

$$\hat{y} = b + w_1 x_1 + w_2 x_1^2 + w_3 x_2 + w_4 x_2^2 + w_5 x_1 x_2$$

This is why the number of generated columns grows fast. For *n* input features and degree *d*, `PolynomialFeatures` generates every combination of powers whose exponents add up to at most *d*. In general, the total degree of a term is simply the sum of the powers of every feature inside it.



Two input features + degree-2 polynomial terms produce a curved surface (instead of a curved line).

Practical tip: with more than a handful of features, keep the degree low (2, sometimes 3). The column count explodes quickly and overfitting risk grows right along with it.

10 · Finding the Best Degree — Pro Workflow

Here is the workflow experienced practitioners actually use, pieced together from how it's done end-to-end:

Step 1 — Split your data

Hold out a test set (or use several folds via cross-validation) so you can measure performance on data the model never trained on.

Step 2 — Loop over candidate degrees

```
train_error, test_error = {}, {}

for d in range(1, 11):
    poly  = PolynomialFeatures(degree=d)
    Xtr_p = poly.fit_transform(X_train)
    Xte_p = poly.transform(X_test)

    model = LinearRegression().fit(Xtr_p, y_train)

    train_error[d] = rmse(y_train, model.predict(Xtr_p))
    test_error[d]  = rmse(y_test,  model.predict(Xte_p))
```

Step 3 — Plot RMSE vs. degree

Plot both dictionaries (see the chart in Section 7). Pick the degree where the testing curve is lowest — not where the training curve is lowest.

Step 4 — Prefer cross-validation for small datasets

A single train/test split can be noisy on small datasets. `k`-fold cross-validation repeats the split several times and averages the error, giving a more reliable estimate of the best degree.

```
from sklearn.model_selection import cross_val_score

for d in range(1, 11):
    poly = PolynomialFeatures(degree=d)
    Xp = poly.fit_transform(X)
    scores = cross_val_score(LinearRegression(), Xp, y,
                             cv=5, scoring="neg_root_mean_squared_error")
    print(d, -scores.mean())
```

Root Mean Squared Error (RMSE) — the metric used above — is defined as:

$$RMSE = \sqrt{\frac{1}{n} \times \sum_{i} (y_i - \hat{y}_i)^2}$$

It measures the average size of the model's prediction mistakes, in the same units as y — smaller is better.

11 · The Math Behind It (Normal Equation)

Once the input has been expanded into polynomial features, solving for the weights is identical to solving ordinary linear regression. Stack every transformed feature row into a matrix X (with a column of 1s for the bias) and every target into a vector y. The weight vector w that minimizes squared error has a closed-form solution:

$$w = (X^T X)^{-1} X^T y$$

This is called the **Normal Equation**. In code (and inside `LinearRegression.fit()` itself) it looks like:

```
import numpy as np

# X: (n_samples, n_poly_features + 1) including a bias column of 1s
w = np.linalg.inv(X.T @ X) @ X.T @ y

y_pred = X @ w
```

In practice, libraries rarely invert the matrix directly (it can be numerically unstable); they use more robust decompositions internally, but the underlying idea is exactly this equation.

Why does this matter for polynomial regression specifically? Because X here is not the raw input — it's the *expanded* input (x, x^2, x^3, …). The model is linear in its weights the whole time; only the input features are non-linear transformations of x. That's the whole trick, from the simplest quadratic curve to a 20-term multi-feature expansion.

12 · Cheat Sheet & Key Takeaways

- **What it is:** linear regression trained on powers of x (x, x-squared, x-cubed, …) so the prediction curve can bend.
- **Why use it:** straight lines underfit curved/non-linear data.
- **How to build it:** `PolynomialFeatures` to expand features, then `LinearRegression` to fit weights.
- **Degree too low** leads to underfitting: high error on both train and test data.
- **Degree too high** leads to overfitting: near-zero train error, high test error.
- **Pick the degree** using the test-error curve (or cross-validation), never the training error alone.
- **Multiple features** add interaction terms (like x1×x2) — total degree = sum of each feature's exponent in that term.
- **Under the hood** it's all still solved with the linear-regression normal equation, w = inverse(X transpose X) × X transpose × y — only the input matrix X changes.

Where to go next

Try this on your own dataset: start at degree 1, plot the fit, then increase the degree one step at a time while watching the train/test RMSE chart from Section 7. The moment the testing curve turns upward again, stop — you've found your degree.

End of guide — happy modeling!